Embeddings & Vector Representations
Embedding Fundamentals
Chinese & Chinese translation

Show more

Let's go deep on embeddings — the mathematical foundation that makes semantic search possible. This is one of the most interview-heavy topics in modern AI/ML roles, at the intersection of theory (linear algebra, information retrieval) and engineering (model selection, latency, cost).

1. Embedding Fundamentals — The Core Intuition

Embeddings are the answer to a single question: *how do you represent meaning as math?* Text in discrete and symbolic "cat" and "feline" are completely different strings, yet they mean the same thing. Cosine distance between their vectors will be nearly zero. Euclidean distance will be large. Embedding model maps both words into a continuous vector space where "cat" and "feline" end up nearby — because they were used in similar contexts across billions of training sentences.

Think of it as getting every word, sentence, or paragraph a GPS coordinate in meaning-space. Two passages about "insurance deductible" will have coordinates that are geographically close. A passage about "car engines" will be far away.

Dense vs Sparse Vectors

This is one of the most fundamental distinctions in the entire domain.

Dense vectors are the output of neural embedding models. Every dimension carries a value — typically floating point, 3-96 dimensional dense vector has 768 non-zero values. The "meaning" is distributed across all dimensions simultaneously. Cannot be directly interpreted. The meaning is encoded holistically by the pattern of all values together.

Sparse vectors are the opposite: most values are zero, with non-zero values only at positions corresponding to tokens present in the text. A document about "insurance deductible" might have non-zero values only at the dimensions for "insurance", "deductible", "plan", "annual". Everything else is 0. This is exactly TF-IDF and BM25 vectors.

Think of it as getting every word, sentence, or paragraph a GPS coordinate in meaning-space. Two passages about "insurance deductible" will have coordinates that are geographically close. A passage about "car engines" will be far away.

Similarity Metrics — The Three You Must Know

Once you have vectors, you need a way to compare them. Three metrics dominate:

Cosine similarity measures the angle between two vectors, ignoring their magnitudes. Formula: `cos(A, B) = A · B / (||A|| × ||B||)`. Ranges: `1` = identical direction, `0` = orthogonal (unrelated), `-1` = opposite. This is the default for most embedding models because it's magnitude-invariant — a long document and a short document about the same topic point in the same direction, even though their vectors have different lengths.

Dot product is `A · B`. It combines both angle and magnitude. Faster than cosine but only equivalent to cosine when vectors are normalized to unit length. Ada models output normalized vectors, so dot product equals cosine similarity. Many models train to maximize dot product on normalized vectors, so in practice they're equivalent. Computationally faster than cosine (skip the division), which matters at billion-vector scale.

Euclidean distance is the straight-line distance between two points: `||A - B||`. Ranges: `0` = identical, `∞`. Lower = more similar. Works well in low-dimensional spaces, but suffers from the "curse of dimensionality" in high-dimensional spaces (everything becomes roughly equidistant). Rarely used for semantic similarity in high-dimensional embedding spaces.

Which to use? Always check your model's training objective. Models trained with cosine loss → use cosine. Models trained with dot product (most OpenAI models, after normalization) → use dot product. Default to cosine similarity for any new model unless the docs say otherwise. In practice for normalized vectors it doesn't matter.

Normalization means scaling a vector to it has unit length (`||A|| = 1`). After normalization, dot product equals cosine similarity. Ada models output pre-normalized vectors, so you can skip normalization. Saves one computation per comparison. Small optimization but it adds up at scale.

Embedding dimension determines the resolution of the meaning space. More dims = finer-grained representations that can separate more subtle semantic differences. The cost: more storage, more compute, slower search. Ada-002 outputs 1536 dims. `text-embedding-3-large` outputs 3072 dims. `text-embedding-3-small` outputs 1536 dims. Matryoshka models let you truncate to any dimension you want — MTEB RoPE-base = 768, BGE-large = 1024. The sweet spot for most production RAG is 768-1024.

Play with this interactive. Notice how cosine similarity only changes when the angle between vectors changes — it's unaffected by magnitude. The dot product changes with both angle and length. This is why cosine is preferred for retrieval: a longer document shouldn't score higher just because it's longer.

2. Embedding Models — The Production Landscape

Every model makes different trade-offs. Know each one's strengths and when to reach for it.

OpenAI `text-embedding-3`
API hosted

```
Text (max: 8191 tokens) text-embed quality: Best support
Dedicated embeddings — you can truncate to lower dimensions (e.g. 1536 dims) without quality loss, enabling massive storage savings. Strong multilingual support. Go to choose
when: Production RAG where you want zero infrastructure overhead. Pay per token. Latency
depends on network round-trip.
text-3 ada-002 text-3 large 0K-128K tokens 1 2K 3K 3072
```

Cohere `embed-v3`
API hosted

```
Strong performance, the best multilingual embeddings — trained on 100+ languages. Offers
hosted fine-tuning. Supports whether you're embedding a document corpus or a short query, and the
model optimizes differently for each. Go to choose when: Multilingual corpus, or when you want
managed fine-tuning without hosting infrastructure.
text-3 ada-002 text-3 large 0K-128K tokens 1 2K 3K 3072
```

Voyage AI `voyage-3`
API hosted

```
Consistently ranks at the top of MTEB benchmarks. Domain-specific models available,
including voyage-code-2 for code and voyage-law-2 for legal text. Long context windows
(16K tokens). Strong for code-heavy or technical corpus. Go to choose when: You need
state-of-the-art retrieval quality, or your domain is specialized enough that general models
underperform.
text-3 ada-002 text-3 large 0K-128K tokens 1 2K 3K 3072
```

BGE (BAAI) `bge-base-en-v1.5`
Open-source, self-hosted

```
BGE MTEB is arguably the most capable open-source embedding model. Fully self-
hostable, no API dependency, no data leaving your infrastructure. Requires GPU for
speed. This makes it a great step up from MiniLM for teams that need quality
improvements but can't use cloud APIs. Go to choose when: Regulated industries (HIPAA,
GDPR), or high-volume RAG where API costs would be prohibitive.
Model Dim. ONNX ARM 1 quant Size. CPU. GPU. 256 384 768 1024
```

E5 (Microsoft) `e5-base-v2`
Open-source, self-hosted

```
E5 models require a specific prefix before passing text to the model: `query: ` for
search queries and `passage: ` for the documents being retrieved. Forgetting this
degrades performance significantly. Go to choose when: Open-source RAG where you
control the entire pipeline and can handle the prefix convention.
Model Dim. ONNX ARM 1 quant Size. CPU. GPU. 256 384 768 1024
```

Jina Embeddings `jina-embeddings-v2`
Open-source

```
Supports long-context inputs — 8K tokens by default, extendable to 32K. Useful when
chunks are large or when you want to embed entire sections without aggressive chunking.
Good for legal documents, research papers, or any domain where context matters.
Model Dim. ONNX ARM 1 quant Size. CPU. GPU. 256 384 768 1024
```

3. Sparse Embeddings — Lexical Retrieval

Sparse vectors rule next. It's resurging in hybrid systems because dense retrieval has a critical blind spot: it struggles with rare keyword matching, especially for rare or technical terms not well represented in training data.

TF-IDF

Term Frequency-Inverse Document Frequency. The classic. TF counts how often a term appears in a document (normalized by length). IDF penalizes terms that appear in many documents (common words like "the" get near-zero weight). The product gives each term a weight that reflects how distinctive it is for that document.

```
TF(term, doc) = count(term in doc) / total_terms(doc)
IDF(term) = log(total_docs / docs_containing_term)
TF-IDF = TF × IDF
```

The resulting vector has one dimension per vocabulary term (~30,000 dims for English). Most are zero. Cosine similarity over TF-IDF vectors is fast lexical search.

BM25 — The Production Standard for Sparse Retrieval

BM25 (Best Match 25) is a probabilistic extension of TF-IDF with two key improvements:

Term frequency saturation: repeated terms get diminishing returns. The 10th occurrence of "deductible" is a document adds far less signal than the 1st. BM25 uses a saturation curve instead of linear TF.

Document length normalization: shorter documents are favored. A 200-word document with "deductible" once is more focused than a 5000-word document with the same term.

```
BM25(q, D) = Σ IDF(q_i) × f(q_i, D) × (k1+1) / (f(q_i, D) + k1 × (1 - b + b × |D| / avgdl))
```

The `k1` parameter (typically 1.2-2.0) controls saturation speed. `b` (typically 0.75) controls length normalization strength. BM25 consistently outperforms TF-IDF on retrieval benchmarks and is the default in Elasticsearch, OpenSearch, and most information retrieval systems.

BM42 is a recent refinement from Qdrant that replaces the IDF component with attention weights from transformer models, combining the interpretability of BM25 with learned importance signals. Early results show improvements on BEIR.

SPLADE — Neural Sparse Models

SPLADE (Sparse Lexical and Expansion) is where sparse and dense models start merging. It's a BERT-based model trained to produce sparse vectors — but unlike TF-IDF, it also performs query expansion. When you embed "car insurance", SPLADE might activate not just "car" and "insurance" but also related terms like "vehicle", "automobile", "premium", even if they don't appear in the text. The model learns what terms to expand to maximize retrieval accuracy, while keeping the sparsity benefits.

SPLADE vectors are interpretable — you can look at the highest weight terms and understand what the model "thinks" the document is about.

4. Hybrid Embeddings — Advanced Representations

These are the cutting-edge techniques that appear in senior interview questions.

ColBERT — Late Interaction

Standard bi-encoder: one vector per document, one vector per query, one dot product. Fast but lossy — the document vector must compress everything into a single vector before any query is known.

ColBERT takes a different approach: keep ALL token level vectors for both query and document. At query time, for each query token, find its maximum similarity to any document token. Sum the maximum similarity scores. All dot products, no learned weights.

```
ColBERT_score(Q, D) = Σ_i max_j (Q_i · D_j)
```

Where `Q_i` is the embedding of the i-th query token and `D_j` is the j-th document token's embedding.

This "late interaction" model is significantly more accurate than single vector retrieval because it can match at token granularity. The cost: much larger storage (100 vectors per document instead of 1) and slower scoring. Used primarily as a re-ranker (ColBERT v2) rather than first-stage retrieval.

Matryoshka Representation Learning (MRL)

Named after Russian nesting dolls. Trains embedding models to nest embedding research. Traditional models produce fixed dimension vectors — you embed at 1536 and you get 1536. Matryoshka models (OpenAI text-embedding-3 models, MRL) make the model so that the first N dimensions of the vector are themselves a valid embedding at that lower dimensionality.

The training objective: the first 64 dims should give a good 64-dim embedding. The first 128 should be better, 256 better still, 1536 best. Loss function sums over multiple truncations (matryoshka), and smaller embedding is contained within the larger one.

The practical impact: OpenAI's `text-embedding-3` models support MRL. You can embed at 1536 dims and store at 256, with only ~3% quality degradation. Cut storage costs by 83% or more. Enables adaptive retrieval: fast search at low dims, re-rank with full dimensions.

Binary Embeddings

Extreme quantization: each dimension becomes one bit. Positive → 1, Non-positive → 0. The result is 32x smaller than FP32 (a 1536-dim vector becomes 48 bytes instead of 1536 bytes). Hamming distance (count of differing bits) replaces cosine. Blazing fast: XOR + popcount. Surprisingly still usefulness: MTEB retrieval drops only ~5-10% for well-trained models. Use cases: billion-scale first-stage filtering, mobile/edge deployment, cost-sensitive applications. Typically paired with a full-precision re-ranker.

Quantization

Broader than binary: any technique to reduce bits per dimension. Scalar quantization: FP32 → INT8 (4x smaller, ~1% quality loss). Product quantization: compress groups of dimensions jointly. Critical for billion-scale deployments where memory is the bottleneck.

5. Fine-tuning Embeddings — When Generic Models Aren't Enough

Generic models are trained on web-scale general corpora. Your insurance documents, your medical notes, your legal contracts — the vocabulary and relevance patterns are different. Fine-tuning adapts the embedding space to your domain.

Domain Adaptation via Contrastive Continuing

The fundamental training objective for embedding models is contrastive: given a query, pull relevant documents closer in the embedding space and push irrelevant documents further away. This is learned through positive/negative pairs.

The core loss function is InfoNCE (in-batch negatives):

```
L = -log( exp(sim(q, d⁺)/τ) / Σ_j exp(sim(q, d_j)/τ) )
```

Where `q` is query, `d⁺` is the positive (relevant) document, `d_j` are in-batch negatives, and `τ` is a temperature parameter.

Intuition: maximize similarity to the ground truth, minimize similarity to everything else in the batch. For an insurance corpus, a hard negative for "What is deductible?" might be "What is deductible?" — similar vocabulary, different answer.

Synthetic Data Generation for Fine-tuning

The bottleneck is fine-tuning is labeled pairs. You need (query, relevant_document) examples. In most domains these don't exist. The modern solution: generate them synthetically.

```
prompt
# Pick random chunk from your corpus.
# Ask LLM to generate realistic question
chunk = chunks[random_idx]
# Output only the questions, one per line.
questions_raw = llm.generate(prompt)
# Now you have (query, chunk) pairs for training
```

This approach, pioneered by the BGE and E5 papers, produces surprisingly high quality training data. The LLM sees a chunk and asks "what question would this chunk answer?" — exactly the right question for retrieval. Cost: ~$100 for 100K pairs using GPT-3.5-turbo.

MTEB — The Embedding Model Benchmark

The Massive Text Embedding Benchmark evaluates models across 56 datasets and 8 task types: retrieval, classification, clustering, STS, reranking, pair classification, summarization, bitext mining. Retrieval NDCG@10 is the main metric. Leaderboard: huggingface.co/spaces/mteb/leaderboard. Top models (as of 2024): OpenAI text-embedding-3-large, Cohere embed-v3, Voyage-2, BGE-M3 (multilingual), E5-mistral-7b-instruct.

When choosing a model, look at MTEB retrieval scores specifically, not the overall MTEB score which averages across tasks unrelated to RAG. Voyage and BGE lead consistently on retrieval. Open weights: BGE, E5, GTE.

Sentence Transformers Training API

```
from sentence_transformers import SentenceTransformer, InputExample, losses
from torch.utils.data import DataLoader

# Load base model
model = SentenceTransformer('BAAI/bge-base-en-v1.5')

# Build training pairs
train_examples = [
    InputExample(texts=['What is the deductible?', 'Plan A has a $500 annual
deductible.'], label=1.0),
    InputExample(texts=['What is the deductible?', 'Plan A covers vision care'],
label=0.0),
]

# Multiple loss functions available.
# MultipleNegativesRankingLoss - best for (query, passage) pairs
# CosineSimilarityLoss - for (text1, text2, similarity_score)
# TripletLoss - for (anchor, positive, negative) triplets
train_dataloader = DataLoader(train_examples, shuffle=True, batch_size=32)
train_loss = losses.MultipleNegativesRankingLoss(model)

model.fit(train_objectives=[(train_dataloader, train_loss)], epochs=1)
model.save('my-domain-embedding-model')
```

6. Embedding Best Practices — Production Engineering

Choosing the right model

```
Managed API (OpenAI, Cohere, Voyage)
Speed: fast (network round-trip)
Quality: state-of-the-art
Cost: $0.02-0.13 per 1M tokens
Ops: zero infrastructure
Best for: Prototyping, low-volume production, when you don't want to manage infrastructure

Self-hosted open-source (BGE, E5, GTE, Jina)
Speed: depends on your GPU
Quality: competitive with managed
Cost: GPU compute only
Ops: you manage deployment, scaling, updates
Best for: Regulated industries (HIPAA, GDPR), high-volume (API costs prohibitive), when data can't leave your infrastructure

Domain-specific
Speed: same as base model
Quality: outperforms base models on your domain
Cost: training + hosting
Ops: ML pipeline for training, evaluation, deployment
Best for: When general models underperform on your specific vocabulary and you have enough data (~10K+ pairs) to fine-tune
```

Batching strategies

```
Never embed one chunk at a time in a loop — GPU underutilized. Batch 100-500 chunks per request. For OpenAI
API, batch up to 2048 inputs. Watch token limits (8191 per input for text-embedding-3). Implement retry with
exponential backoff for rate limits. Cache results keyed by content hash (hashlib.sha256(text)). Embeddings
are deterministic — same input always produces the same output.
```

```
chunks_with_hashes = [(chunk, hashlib.sha256(chunk.encode())
.hexdigest()) for chunk in chunks]
# Check which hashes are already in cache
cache_hits, cache_misses = cache.lookup(chunks_with_hashes)
```

Caching embeddings

Embeddings are deterministic and expensive to compute. Cache them aggressively. Content-addressable caching: hash the input text, use the hash as the cache key. Invalidation is simple: if the text changes, the hash changes, you get a cache miss. Storage options: Redis (fast, memory-expensive), PostgreSQL with pgvector (persistent, queryable), S3 + FAISS (cheap, slow). Typical cache hit rate in production: 60-80% on stable corpora.

Embedding versioning — the critical production concern

When you upgrade your embedding model, ALL existing vectors become invalid. You can't mix old and new embeddings in the same index — they live in different vector spaces. The migration: re-embed everything, rebuild the index, switch traffic. Plan for this from the start. Store model version alongside each vector. Have a rollback plan. Expect this to take days for large corpora. Blue-green deployment is common: run old and new indexes in parallel, route traffic to new index when ready, keep old index for rollback.

Cost optimization

```
OpenAI text-embedding-3-small: $0.02 per 1M tokens. Cheapest managed option. Quality is
still excellent for most use cases. Start here unless you have specific requirements.

Batch processing: Process overnight or during off-peak hours. Some providers offer
discounts for batch jobs. Not relevant for real-time search, but worth knowing for
bulk re-indexing.

Dimension reduction: Matryoshka models let you use 256 dims instead of 1536. Storage
drops 6x, quality degrades ~3%. Often worth the trade-off at scale.

Quantization: Scalar (INT8) or binary. Combine with dimension reduction for maximum
compression.

Self-hosted: Amortized GPU cost can be cheaper than API at high volume. Calculate your
break-even point.
```

Evaluation

```
Retrieval metrics: Recall@k (did the relevant doc appear in top k?), MRR (mean reciprocal
rank), NDCG (normalized discounted cumulative gain). Ground truth: human-labeled relevance
judgments, or LLM-generated relevance scores (cheaper, noisier).

Tools: Ragas (RAG evaluation framework), custom eval scripts, MTEB benchmark for
comparison against baselines.

Key insight: Embedding quality is measured by downstream retrieval performance, not
intrinsic properties. Always evaluate on your actual retrieval task.
```

Interview Question Bank — Domain 5

Basic What is the difference between dense and sparse embeddings?

Dense embeddings are continuous, high-dimensional vectors where almost every dimension is non-zero — produced by neural models. They capture semantic meaning and synonyms. Sparse embeddings have mostly zero values with non-zero values only at specific token positions — TF-IDF and BM25 are classic sparse representations. Dense excels at "find something similar in meaning" queries. Sparse excels at "find exactly this keyword" queries. Production systems use both: dense for semantic recall, sparse for exact term matching, then combine both to get the benefits of each.

Basic Why use cosine similarity over dot product or Euclidean distance?

Cosine similarity measures the angle between vectors, ignoring magnitude. This matters because you don't want document length to affect relevance — a long document and a short document about the same topic should score similarly. Dot product combines both angle and magnitude. Euclidean distance is sensitive to magnitude and suffers from the "curse of dimensionality" in high-dimensional spaces. For normalized vectors, cosine and dot product are equivalent. Most embedding models are trained with cosine similarity as the training objective, so it's the safe default.

Intermediate Explain BM25 and why it remains a hard baseline.

BM25 is a probabilistic sparse retrieval function that scores documents based on term frequency (TF) and inverse document frequency (IDF), with two key improvements over TF-IDF: term frequency saturation (the 10th occurrence of "deductible" in a document adds far less than the 1st) and document length normalization (shorter documents are favored over longer ones at equal term frequency). It remains a hard baseline because dense embeddings have a systematic blind spot: they struggle with rare keywords, proper nouns, and exact technical terms that weren't well represented in training data. Production systems almost always combine BM25 with dense retrieval in hybrid search.

Intermediate When would you choose a managed API vs self-hosted embedding model?

Managed APIs (OpenAI, Cohere, Voyage) for: prototyping, low-volume production, when you don't want to manage infrastructure, when your data can leave your cloud. Self-hosted (BGE, E5, GTE) for: regulated industries (HIPAA, GDPR), high-volume where API costs would be prohibitive, when latency matters (network round-trip adds 100-200ms), when you need fine-tuning on your domain. Break-even typically at 10-50M tokens/month depending on GPU cost. Self-hosted also gives you control over model versions — managed APIs can change under you.

Intermediate What is Matryoshka Representation Learning and why is it useful?

MRL trains embedding models so that every prefix of the full dimensional vector is itself a valid embedding. The first 64 dims should be a good 64-dim embedding, the first 128 better, 256 better still, 1536 best. This enables adaptive retrieval: use low-dim vectors for fast approximate search, then re-rank with full dimensions for accuracy. Storage savings: OpenAI's text-embedding-3 models support MRL, so you can store at 256 dims instead of 1536 and only lose ~3% quality. Trade-off: training MRL models requires modifying the loss function to sum over multiple truncations, and not all models support it.

Intermediate Why do we need a fine-tuning API for embeddings?

Pre-trained embeddings optimize for general similarity. But your domain may have specialized vocabulary, implicit relevance patterns, or domain-specific definitions of "similar" that differ from what the model learned. Fine-tuning with contrastive pairs (query, relevant_document, hard_negatives) teaches the model your domain's notion of relevance. Typical improvement: 5-15% on retrieval metrics after fine-tuning on 10K+ pairs. Most embedding APIs don't offer this because it requires ML infrastructure they don't want to manage.

Advanced Explain ColBERT late interaction. How does it differ from bi-encoder retrieval?

Bi-encoder: one vector per document, one vector per query, one dot product. Fast but lossy — the document must compress everything into a single vector before any query is known. ColBERT: keep all token level vectors for both query and document. For each query token, find its maximum similarity to any document token. Sum the maximum similarities. More accurate than bi-encoder because it preserves token level information. Cost: much larger storage (100 vectors per document instead of 1) and slower scoring. Used primarily as a reranker (ColBERT v2) rather than first-stage retrieval.

Advanced Why do embedding models struggle with negation? What's the workaround?

Embeddings capture what co-occurs with what. "I love X" and "I hate X" both place X in similar contexts, so their embeddings are nearby. The model learns that negation matters grammatically, but this doesn't translate well to vector distance. Workarounds: (1) Generate hard negatives that include negation during fine-tuning, so the model sees "I love X" paired with "I hate X" as a positive pair. (2) Use a cross-encoder reranker that can attend to the negation token. (3) Hybrid search: BM25 will match "not" exactly, so keyword search handles negation queries better than semantic search.

Advanced Design a system to embed 1B documents. What are the key decisions?

Model: open-weight for control and cost at scale. Quantization: binary for first-stage filtering, full precision for re-ranking. Sharding: by document ID, with replication for fault tolerance. Index: HNSW for speed, IVF for memory efficiency. Refresh: incremental indexing for new documents, periodic full rebuild to clean up deletions. Monitoring: track recall@k against a held-out evaluation set, latency p99, queries per second. Cost: ~$100K/month for managed API, ~$20K/month for self-hosted at this scale.

The Mental Models That Win Interviews

Embeddings = GPS coordinates for meaning. You choose about instance distributions here coordinates that are geographically close. A chunk about heavy is far away. Retrieval is just "find nearest neighbors in this GPS space".

Dense for semantics, sparse for exact match, hybrid for production. Dense finds similar meaning. Sparse finds exact terms. The hybrid wins because "deductible" and "car insurance" are both far apart in GPS but are not found by the other's keyword.

Cosine = angle, dot product = angle + magnitude. The direction of a document and the length of a document. Cosine ignores length, dot product includes it. Normalize vectors, then they're equivalent. Ada models do this for you.

MRL = slide the embedding. You can decompose by any dim. Full quality at 1536, 256 dims, 64 dims. Matryoshka nesting. Trade dimensions for storage and speed.

Embedding versioning = distance between regions. You upgrade from ada-002 to text-embedding-3-small. Your old vectors live in one region, your new embeddings in another. Cosine similarity between vectors from different models is meaningless. Re-embed everything or don't upgrade.

Ready for Domain 6: Vector Databases & Indexing?

Reply...

Gemini 3.1 Pro